Recursion

Content notes, classwork and homework with worked C++ solutions.

Lesson 1: Recursion - Intro

Quick recap: Base case and recursive case • The call stack • factorial, power, sum recursively • Thinking recursively

Theory & Nuances

What is recursion?

Recursion is when a function calls itself to solve a smaller copy of the same problem. Instead of writing a loop, you describe the answer in terms of a smaller answer: 5! is 5 * 4!, and 4! is 4 * 3!, and so on. Every recursive function has exactly two parts, and forgetting either one breaks it.

Base case and recursive case

The base case is the smallest input whose answer you already know without recursing - factorial(1) = 1, sum of digits of 0 = 0, a^0 = 1. It is the stopping point and is usually written first. The recursive case calls the same function on a SMALLER input (n - 1, n / 10, b / 2) and uses that result to build the answer. Each call must move closer to the base case; if it does not, the calls never stop.

The call stack

Each call gets its own box (a stack frame) holding its own copy of the parameters and local variables. When factorial(5) calls factorial(4), factorial(5) is paused and waits on the stack. The calls pile up until the base case returns, then they finish in reverse order - the last call made is the first to return (LIFO). Code written BEFORE the recursive call runs on the way down; code written AFTER it runs on the way back up. That is the only difference between printing 1..N and N..1.

factorial, power, sum recursively

factorial(n) = n * factorial(n - 1) with factorial(0 or 1) = 1. power(a, b) = a * power(a, b - 1) with power(a, 0) = 1 - this makes b calls. Fast power halves b instead: compute half = power(a, b / 2) ONCE, then return half * half (times a extra if b is odd), which needs only about log2(b) calls. sum(n) = n + sum(n - 1) with sum(0) = 0, and the sum of digits is n % 10 + sumOfDigits(n / 10).

Thinking recursively (the leap of faith)

Do not try to trace every call in your head. Use three steps: (1) write the base case, (2) ASSUME the function already works correctly for the smaller input, (3) use that smaller answer to build the answer for n. If sumOfDigits(9876) is trusted to return 30, then sumOfDigits(98765) is just 5 + 30. Trace small inputs on paper only to check your work.

Stack overflow

Every waiting call uses stack memory, so recursion depth N costs O(N) extra space. A missing or unreachable base case (e.g. calling factorial(n) instead of factorial(n - 1), or factorial(-3) when the base case only checks n == 1) makes calls pile up forever until the program crashes with a stack overflow / segmentation fault.

Key Points / Common Mistakes

- Always write the base case first, and make sure every input eventually reaches it.
- Each recursive call must work on a smaller input: n - 1, n / 10, b / 2 - never n itself.
- Use n <= 1 instead of n == 1 as the factorial base case so 0 and wrong inputs still stop.
- Statements before the recursive call run going down; statements after it run coming back up.
- Recursion depth N uses O(N) stack space. Factorials overflow int quickly - use long long.
- Fast power: call power(a, b / 2) once and store it. Calling it twice makes it slow again.

Classwork Problems

1. Factorial recursively

```cpp
#include <iostream>
using namespace std;

long long factorial(int n) {
    cout << "Called factorial(" << n << ")" << endl;
    if (n <= 1) {                               // base case
        cout << "Base case: factorial(" << n << ") returns 1" << endl;
        return 1;
    }
    long long result = n * factorial(n - 1);     // recursive case
    cout << "factorial(" << n << ") returns " << result << endl;
    return result;
}

int main() {
    int n = 5;
    long long answer = factorial(n);
    cout << n << "! = " << answer << endl;
    return 0;
}
```

Output

```
Called factorial(5)
Called factorial(4)
Called factorial(3)
Called factorial(2)
Called factorial(1)
Base case: factorial(1) returns 1
factorial(2) returns 2
factorial(3) returns 6
factorial(4) returns 24
factorial(5) returns 120
5! = 120
```

2. Print 1..N and N..1 recursively

```cpp
#include <iostream>
using namespace std;

void printIncreasing(int n) {
    if (n == 0) return;          // base case: nothing left to print
    printIncreasing(n - 1);      // first print 1 .. n-1
    cout << n << " ";            // then print n (on the way back)
}

void printDecreasing(int n) {
    if (n == 0) return;
    cout << n << " ";            // print n first (on the way down)
    printDecreasing(n - 1);      // then print n-1 .. 1
}

int main() {
    int N = 5;
    cout << "1 to N: ";
    printIncreasing(N);
    cout << endl;
    cout << "N to 1: ";
    printDecreasing(N);
    cout << endl;
    return 0;
}
```

Output

```
1 to N: 1 2 3 4 5
N to 1: 5 4 3 2 1
```

3. Sum of digits recursively

```
#include <iostream>
using namespace std;

int sumOfDigits(int n) {
    if (n == 0) return 0;                    // no digits left
    return n % 10 + sumOfDigits(n / 10);     // last digit + sum of the rest
}

int main() {
    int a = 98765, b = 1004, c = 7;
    cout << "Sum of digits of " << a << " = " << sumOfDigits(a) << endl;
    cout << "Sum of digits of " << b << " = " << sumOfDigits(b) << endl;
    cout << "Sum of digits of " << c << " = " << sumOfDigits(c) << endl;
    return 0;
}
```

Output

```
Sum of digits of 98765 = 35
Sum of digits of 1004 = 5
Sum of digits of 7 = 7
```

Homework Problems

1. Fast power a^b

```
#include <iostream>
using namespace std;

int calls = 0;   // counts how many times power() runs

long long power(long long a, int b) {
    calls++;
    if (b == 0) return 1;                  // a^0 = 1
    long long half = power(a, b / 2);      // solve the half problem ONCE
    if (b % 2 == 0) return half * half;    // even: a^b = (a^(b/2))^2
    return half * half * a;                // odd: one extra a
}

int main() {
    calls = 0;
    cout << "2^10 = " << power(2, 10) << " (calls made: " << calls << ")" << endl;
    calls = 0;
    cout << "3^5 = " << power(3, 5) << " (calls made: " << calls << ")" << endl;
    calls = 0;
    cout << "2^30 = " << power(2, 30) << " (calls made: " << calls << ")" << endl;
    calls = 0;
    cout << "7^0 = " << power(7, 0) << " (calls made: " << calls << ")" << endl;
    return 0;
}
```

Output

```
2^10 = 1024 (calls made: 5)
3^5 = 243 (calls made: 4)
2^30 = 1073741824 (calls made: 6)
7^0 = 1 (calls made: 1)
```

2. Recursive palindrome check

```
#include <iostream>
using namespace std;

bool isPalindrome(string s, int left, int right) {
    if (left >= right) return true;              // 0 or 1 characters left
    if (s[left] != s[right]) return false;       // outer pair mismatch
    return isPalindrome(s, left + 1, right - 1); // check the inside
}

void check(string word) {
    bool ok = isPalindrome(word, 0, word.length() - 1);
    cout << word << (ok ? " is" : " is not") << " a palindrome" << endl;
}

int main() {
    check("racecar");
    check("level");
    check("hello");
    check("a");
    return 0;
}
```

Output

```
racecar is a palindrome
level is a palindrome
hello is not a palindrome
a is a palindrome
```

Lesson 2: Recursion - Practice

Quick recap: Fibonacci • Subsequences of a string • Reading a recursion tree • Recursion on arrays and strings

Theory & Nuances

Fibonacci - more than one recursive call

The Fibonacci series is 0, 1, 1, 2, 3, 5, 8, 13 ... where each term is the sum of the two before it: fib(n) = fib(n - 1) + fib(n - 2), with two base cases fib(0) = 0 and fib(1) = 1. Because each call makes TWO calls, the work branches out like a tree instead of a straight line.

Reading a recursion tree

Draw each call as a node and its recursive calls as its children. The root is the first call, the leaves are base cases. The program explores the tree depth-first: it goes all the way down the LEFT child before starting the right one. The number of nodes is the number of calls (time), and the height of the tree is the maximum stack depth (space). For fib(n) the tree has roughly 2^n nodes but height only n, so naive Fibonacci is O(2^n) time and O(n) space. Notice fib(2) is computed twice inside fib(4) - this repeated work is why it is slow (it is fixed later with memoization / DP).

Subsequences of a string - take or skip

A subsequence keeps some characters in their original order and drops the rest: "ac" is a subsequence of "abc", but "ca" is not. For each character you have exactly two choices: TAKE it or SKIP it. So the function carries an index and the string built so far; it calls itself once with the character added and once without. When index reaches the end, one complete subsequence is printed. n characters give 2 * 2 * ... * 2 = 2^n subsequences, including the empty one.

Recursion on arrays and strings

You cannot shrink an array, so pass its size (or a starting index) and shrink that instead: sum(arr, n) = arr[n - 1] + sum(arr, n - 1), with sum(arr, 0) = 0. Strings work the same way with an index that moves forward (index + 1) until it equals s.length(), or with two indices (left, right) that move towards each other, as in the palindrome check and in-place reverse.

Key Points / Common Mistakes

- Fibonacci needs two base cases (n = 0 and n = 1); with only one, fib(1) would call fib(-1).
- Naive fib(40) makes over 300 million calls - recursion trees grow exponentially when calls branch.
- Subsequence order matters: the characters keep their original order. Substring = contiguous; subsequence need not be.
- Index-based recursion stops when index == length, not when index == length - 1.
- Passing string by value copies it on every call; pass by reference (string &s) to modify the original.

Classwork Problems

1. Nth Fibonacci number

```
#include <iostream>
using namespace std;

// Prints every call, indented by its depth, to draw the recursion tree.
int fibTree(int n, int depth) {
    for (int i = 0; i < depth; i++) cout << "    ";
    cout << "fib(" << n << ")" << endl;
    if (n <= 1) return n;
    return fibTree(n - 1, depth + 1) + fibTree(n - 2, depth + 1);
}

int fib(int n) {
    if (n <= 1) return n;                 // fib(0) = 0, fib(1) = 1
    return fib(n - 1) + fib(n - 2);       // two recursive calls
}

int main() {
    cout << "Recursion tree for fib(4):" << endl;
    int ans = fibTree(4, 0);
    cout << "fib(4) = " << ans << endl;

    cout << "First 10 Fibonacci numbers: ";
    for (int i = 0; i < 10; i++) {
        cout << fib(i) << " ";
    }
    cout << endl;
    return 0;
}
```

Output

```
Recursion tree for fib(4):
fib(4)
    fib(3)
        fib(2)
            fib(1)
            fib(0)
        fib(1)
    fib(2)
        fib(1)
        fib(0)
fib(4) = 3
First 10 Fibonacci numbers: 0 1 1 2 3 5 8 13 21 34
```

2. All subsequences of a string

```
#include <iostream>
using namespace std;

int total = 0;

void printSubsequences(string s, int index, string current) {
    if (index == s.length()) {            // made a choice for every character
        cout << "\"" << current << "\"" << endl;
        total++;
        return;
    }
    printSubsequences(s, index + 1, current + s[index]);   // TAKE s[index]
    printSubsequences(s, index + 1, current);              // SKIP s[index]
}

int main() {
    string s = "abc";
    cout << "All subsequences of " << s << ":" << endl;
    printSubsequences(s, 0, "");
    cout << "Total = " << total << " (2^" << s.length() << ")" << endl;
    return 0;
}
```

Output

```
All subsequences of abc:
"abc"
"ab"
"ac"
"a"
"bc"
"b"
"c"
""
Total = 8 (2^3)
```

3. Sum of an array recursively

```
#include <iostream>
using namespace std;

int sumArray(int arr[], int n) {
    if (n == 0) return 0;                         // empty array
    return arr[n - 1] + sumArray(arr, n - 1);    // last element + sum of the rest
}

int main() {
    int arr[] = {3, 8, 1, 9, 4};
    int n = 5;
    cout << "Array: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;
    cout << "Sum = " << sumArray(arr, n) << endl;
    return 0;
}
```

Output

```
Array: 3 8 1 9 4
Sum = 25
```

Homework Problems

1. Count vowels recursively

```
#include <iostream>
using namespace std;

bool isVowel(char c) {
    if (c >= 'A' && c <= 'Z') c = c + 32;     // make it lowercase
    return c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u';
}

int countVowels(string s, int index) {
    if (index == s.length()) return 0;             // reached the end
    int here = isVowel(s[index]) ? 1 : 0;
    return here + countVowels(s, index + 1);
}

int main() {
    string a = "Recursion is Awesome";
    string b = "rhythm";
    cout << "Vowels in \"" << a << "\" = " << countVowels(a, 0) << endl;
    cout << "Vowels in \"" << b << "\" = " << countVowels(b, 0) << endl;
    return 0;
}
```

Output

```
Vowels in "Recursion is Awesome" = 9
Vowels in "rhythm" = 0
```

2. Reverse a string recursively

```
#include <iostream>
using namespace std;

// Way 1: build a new string - reverse(rest) + first character
string reverseString(string s) {
    if (s.length() <= 1) return s;
    return reverseString(s.substr(1)) + s[0];
}

// Way 2: swap the outer pair in place, then reverse the inside
void reverseInPlace(string &s, int left, int right) {
    if (left >= right) return;
    char temp = s[left];
    s[left] = s[right];
    s[right] = temp;
    reverseInPlace(s, left + 1, right - 1);
}

int main() {
    cout << "hello -> " << reverseString("hello") << endl;
    cout << "recursion -> " << reverseString("recursion") << endl;

    string word = "DSA";
    reverseInPlace(word, 0, word.length() - 1);
    cout << "DSA reversed in place -> " << word << endl;
    return 0;
}
```

Output

```
hello -> olleh
recursion -> noisrucer
DSA reversed in place -> ASD
```

Lesson 3: Recursion - Problem Solving

Quick recap: Recursion vs iteration • Divide the range: binary search • Tower of Hanoi • Counting ways • Complexity of recursive code

Theory & Nuances

Recursion vs iteration

Anything you can write with a loop can be written recursively, and vice versa. Loops are usually faster and use O(1) extra space; recursion is often shorter and clearer when the problem naturally splits into smaller copies of itself - trees, choices (take/skip), divide-and-conquer. Pick recursion when it makes the idea obvious, and a loop when a simple counter does the job.

Divide the range: binary search

On a SORTED array, look at the middle element. If it is the target, done. If the target is bigger, it can only be in the right half, so recurse on (mid + 1, high); otherwise recurse on (low, mid - 1). The base case is an empty range (low > high) meaning "not found". Each call throws away half the array, so it takes O(log N) time. Use mid = low + (high - low) / 2 to avoid overflow on huge indices.

Tower of Hanoi

Move n disks from rod A to rod C using rod B, moving one disk at a time and never placing a bigger disk on a smaller one. Leap of faith: (1) move the top n - 1 disks from A to B (using C), (2) move the biggest disk from A to C, (3) move the n - 1 disks from B to C (using A). Base case: 0 disks means do nothing. Moves(n) = 2 * Moves(n - 1) + 1 = 2^n - 1, so 3 disks need 7 moves and 64 disks would take forever.

Counting ways

Many "in how many ways" problems split on the first choice. To climb n stairs taking 1 or 2 steps, the first step is either 1 (leaving n - 1 stairs) or 2 (leaving n - 2), so ways(n) = ways(n - 1) + ways(n - 2). Base cases: ways(0) = 1 (you are at the top - one way, do nothing) and ways(negative) = 0. The answers turn out to be Fibonacci numbers.

Complexity of recursive code

Time = (number of calls) x (work done in one call, not counting the recursive calls). Space = maximum depth of the call stack. One call that shrinks n by 1 -> O(N) time and O(N) space (factorial, sum of array). One call that halves n -> O(log N) (fast power, binary search, Euclid GCD). Two calls that each shrink n by 1 -> O(2^N) (Fibonacci, subsequences, Hanoi).

Key Points / Common Mistakes

- Binary search only works on a sorted array; its base case is low > high, not low == high.
- Hanoi: the helper rod changes in each recursive call - trace the (from, to, helper) order carefully.
- Counting problems need a base case for "done" (return 1) and for "invalid" (return 0).
- Euclid: gcd(a, b) = gcd(b, a % b) and gcd(a, 0) = a. Then LCM = a / gcd * b (divide first to avoid overflow).
- If a recursive solution recomputes the same calls again and again, it is a hint to use memoization later.

Classwork Problems

1. Check if an array is sorted

```
#include <iostream>
using namespace std;

bool isSorted(int arr[], int n) {
    if (n <= 1) return true;                // 0 or 1 element is always sorted
    if (arr[n - 2] > arr[n - 1]) return false;  // last pair out of order
    return isSorted(arr, n - 1);            // check the first n-1 elements
}

int main() {
    int a[] = {1, 3, 5, 7, 9};
    int b[] = {1, 4, 2, 8};
    cout << "{1, 3, 5, 7, 9} is " << (isSorted(a, 5) ? "sorted" : "not sorted") << endl;
    cout << "{1, 4, 2, 8} is " << (isSorted(b, 4) ? "sorted" : "not sorted") << endl;
    return 0;
}
```

Output

```
{1, 3, 5, 7, 9} is sorted
{1, 4, 2, 8} is not sorted
```

2. Binary search recursively

```
#include <iostream>
using namespace std;

int binarySearch(int arr[], int low, int high, int target) {
    if (low > high) return -1;                  // range is empty: not found
    int mid = low + (high - low) / 2;
    cout << "  looking at index " << mid << " (value " << arr[mid] << ")" << endl;
    if (arr[mid] == target) return mid;
    if (arr[mid] < target) return binarySearch(arr, mid + 1, high, target);  // go right
    return binarySearch(arr, low, mid - 1, target);                          // go left
}

int main() {
    int arr[] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
    int n = 10;

    cout << "Searching for 23:" << endl;
    int pos = binarySearch(arr, 0, n - 1, 23);
    cout << "Found at index " << pos << endl;

    cout << "Searching for 7:" << endl;
    pos = binarySearch(arr, 0, n - 1, 7);
    cout << "Result: " << pos << " (not found)" << endl;
    return 0;
}
```

Output

```
Searching for 23:
  looking at index 4 (value 16)
  looking at index 7 (value 56)
  looking at index 5 (value 23)
Found at index 5
Searching for 7:
  looking at index 4 (value 16)
  looking at index 1 (value 5)
  looking at index 2 (value 8)
Result: -1 (not found)
```

3. Tower of Hanoi

```
#include <iostream>
using namespace std;

int moves = 0;

void hanoi(int n, char from, char to, char helper) {
    if (n == 0) return;                     // no disks to move
    hanoi(n - 1, from, helper, to);         // 1. park n-1 disks on the helper
    moves++;
    cout << "Move disk " << n << " from " << from << " to " << to << endl;  // 2. move the biggest
    hanoi(n - 1, helper, to, from);         // 3. bring n-1 disks on top of it
}

int main() {
    int n = 3;
    hanoi(n, 'A', 'C', 'B');
    cout << "Total moves = " << moves << " (2^" << n << " - 1)" << endl;
    return 0;
}
```

Output

```
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
Total moves = 7 (2^3 - 1)
```

Homework Problems

1. Climbing stairs - count the ways

```
#include <iostream>
using namespace std;

// Ways to climb n stairs taking 1 or 2 steps at a time
int countWays(int n) {
    if (n == 0) return 1;     // reached the top exactly: 1 way
    if (n < 0) return 0;      // jumped past the top: not a valid way
    return countWays(n - 1) + countWays(n - 2);
}

int main() {
    for (int n = 1; n <= 6; n++) {
        cout << "Stairs = " << n << " -> " << countWays(n) << " ways" << endl;
    }
    return 0;
}
```

Output

```
Stairs = 1 -> 1 ways
Stairs = 2 -> 2 ways
Stairs = 3 -> 3 ways
Stairs = 4 -> 5 ways
Stairs = 5 -> 8 ways
Stairs = 6 -> 13 ways
```

2. GCD using Euclid's algorithm

```
#include <iostream>
using namespace std;

int gcd(int a, int b) {
    cout << "gcd(" << a << ", " << b << ")";
    if (b == 0) {                    // gcd(a, 0) = a
        cout << endl;
        return a;
    }
    cout << " -> ";
    return gcd(b, a % b);            // gcd(a, b) = gcd(b, a % b)
}

int main() {
    int a = 48, b = 18;
    int g = gcd(a, b);
    cout << "GCD of " << a << " and " << b << " = " << g << endl;
    cout << "LCM of " << a << " and " << b << " = " << a / g * b << endl;

    g = gcd(17, 5);
    cout << "GCD of 17 and 5 = " << g << endl;
    return 0;
}
```

Output

```
gcd(48, 18) -> gcd(18, 12) -> gcd(12, 6) -> gcd(6, 0)
GCD of 48 and 18 = 6
LCM of 48 and 18 = 144
gcd(17, 5) -> gcd(5, 2) -> gcd(2, 1) -> gcd(1, 0)
GCD of 17 and 5 = 1
```